## Immutability

Immutability refers to whether a piece of data can be changed once it's created. The previous section showed how maps are immutable, but this applies to all data types in Elixir.

Immutability also applies across function boundaries. Let's write some code to demonstrate this. Create lib/examples/modules/immutable.ex and enter the following code to see immutability in action:

elixir_examples/lib/examples/modules/immutable.ex

```
defmodule Examples.Modules.Immutable do
  def map_change(map) do
    Map.merge(map, %{a: 1, b: 2})
  end
end
```

```
$ iex -S mix
iex> map = %{number: 1}

iex> Examples.Modules.Immutable.map_change(map)
%{a: 1, b: 2, number: 1}

iex> map
%{number: 1}
```

We passed the map variable into map_change/1, but the original map didn't change.

Immutability makes it significantly easier to reason about data as it's passed around because we know that a variable's data cannot change unless we reassign the variable.

Immutability is a bit of a double-edged sword, though. In Ruby, you can directly modify hashes, lists, or object values. In Elixir, you must iterate and rebuild the data structure to change it.

The best solution to this—and most other things—is practice. As you learn how to effectively use Enum and Map functions, you'll become more comfortable with immutability. We'll dig further into data enumeration later in this chapter.

## Structs

You can use maps to store all of your key-value data, but they are missing a few conveniences. Maps aren't named, so you can't tell the purpose of a map without inspecting the contents. Maps don't have the concept of required keys or default values, which means you would have to implement protection